

TMC-103

OPERATING SYSTEM

M.C.A. (First Semester) — Mid Semester Examination, 2021-22

Fully Solved Question Paper & Revision Notes

Primary Reference:

Operating System Concepts, 9th Edition

Abraham Silberschatz · Peter Baer Galvin · Greg Gagne

How to use this document

The original paper allows you to attempt **only one** sub-part — (a) *or* (b) — of each numbered question, for 10 marks per question (50 marks total). Because the goal here is complete syllabus revision and not just exam attempt, **both sub-parts of every question are answered in full** below, so this single document doubles as revision notes for the whole Mid-Semester syllabus: Process Management, CPU Scheduling, Threads, Shells & System Calls, and Operating System Structures.

Time: 1:30 Hours | Maximum Marks: 50 | Course Outcomes covered: CO1, CO2

CONTENTS

Q. No.	Topics Covered	CO
Q1	Bootting & OS Services / Kernel Structures (Monolithic, Layered, Microkernel, Modules, Hybrid)	CO1
Q2	Priority Scheduling with I/O bursts (numerical) / Round Robin & Priority Scheduling — 5 batch jobs (numerical)	CO2
Q3	Shell Features, Internal vs External Commands, System Call vs Library Function / CPU Utilisation in Round Robin (numerical)	CO1/CO2
Q4	Thread vs Process / Dispatcher & Context Switch	CO1/CO2
Q5	Multilevel Queue vs Multilevel Feedback Queue Scheduling / OS functions: Process, Memory, File, I/O Management	CO1/CO2

Each question below is reproduced exactly as it appears in the original mid-semester paper (with its CO tag), immediately followed by a complete, exam-ready answer for every sub-part.

QUESTION 1

CO1 • 10 Marks

1. (a) How is the operating system loaded in the memory? List the services provided by the operating system.

ANSWER

Part 1 — How the Operating System is Loaded into Memory (Bootting)

The process of starting a computer by loading the kernel into main memory is called **booting**, and the initial program that performs this task is called the **bootstrap program** (or bootstrap loader).

- **Power-on and firmware:** When the system is switched on, control first goes to a small piece of firmware stored in non-volatile memory (ROM/EEPROM) — the BIOS in older PCs, or UEFI firmware in modern systems — because RAM is empty and unreliable at power-up.
- **POST (Power-On Self-Test):** The firmware first checks that the CPU, memory and basic peripherals are functioning correctly.
- **Locating the bootstrap code:** The firmware knows one fixed place to look for further instructions — historically the boot sector / Master Boot Record (MBR) of the designated boot device, or the EFI System Partition for UEFI systems.
- **Two-stage loading:** Because the ROM bootstrap program is deliberately kept tiny, it does not load the full kernel itself. Instead it loads a slightly larger **second-stage boot loader** (e.g., GRUB on Linux, Windows Boot Manager) from disk into memory.
- **Loading the kernel:** This boot loader locates the actual operating-system kernel image on secondary storage, reads it into RAM, sets up an initial execution environment (stack, minimal page tables, etc.) and transfers control to the kernel's entry point.
- **Kernel initialisation:** The kernel then initialises its own data structures, detects and initialises devices via drivers, starts the first user-level process (*init* / *systemd* on UNIX-like systems), which in turn starts the remaining system services, eventually presenting the user with a login prompt or GUI.
- **Diskless / network booting:** Where there is no local boot device, the bootstrap program instead fetches a boot image over the network (e.g., using PXE) from a boot server — common in diskless workstations and thin clients.

Part 2 — Services Provided by an Operating System

The OS provides an environment for the execution of programs and offers common services to programs and users. These services are usually grouped into two categories, as in *Silberschatz, Operating System Concepts, 9th Ed., Ch. 2*:

Group	Service	What it does
Helpful to the user	User Interface	CLI, GUI, batch, or touch interface through which the user issues commands.
	Program Execution	Load a program into memory, run it, and handle normal / abnormal termination.
	I/O Operations	Provide a uniform, controlled way for programs to perform input/output on files or devices.
	File-System Manipulation	Create, delete, read, write, search files/directories; manage permissions.
	Communications	Exchange information between processes on the same machine (shared memory / message passing) or over a network.

Group	Service	What it does
	Error Detection	Detect and, where possible, correct errors in the CPU, memory, I/O devices, or in user programs (e.g., divide-by-zero, illegal memory access).
Helpful to the system (efficiency & sharing)	Resource Allocation	Allocate CPU cycles, main memory, storage and I/O devices among multiple users/jobs.
	Accounting	Keep track of which users use how much and what kind of resources, for statistics or billing.
	Protection & Security	Ensure all access to system resources is controlled, and that processes cannot interfere with one another or with the OS.

1. (b) Enlist the various kernel structure designs and describe each one along with the advantages and disadvantages.

ANSWER

The **kernel** is the core of the OS that runs in privileged (kernel) mode. How its internal components are organised defines the **kernel structure / OS structure**. The commonly studied designs are:

1. Monolithic (Simple) Structure

The entire operating system runs as a single large program in kernel mode, with all functions (process management, memory management, file systems, drivers) interacting freely and directly, without enforced internal boundaries. Example: original UNIX, MS-DOS.

- **Advantage:** Very fast, since any component can call any other directly without message-passing or mode-switch overhead; simple to design initially.
- **Disadvantage:** Difficult to implement, extend and maintain as the OS grows; poor fault isolation — a bug or crash in one component (e.g., a driver) can bring down the entire system; hard to enforce security boundaries internally.

2. Layered Approach

The OS is divided into a number of layers (levels), each built on top of the one below it. The bottom layer (layer 0) is the hardware, the top-most layer is the user interface. Each layer uses only the functions and services of the layer directly beneath it.

- **Advantage:** Strong modularity — construction and debugging are simplified because each layer can be implemented and tested using only the (already verified) services of lower layers.
- **Disadvantage:** Careful, non-trivial layer definition is required because not all functions fit a strict hierarchy (e.g., a device driver may be needed by many layers); each layer adds function-call/data-passing overhead, so it is less efficient than a monolithic kernel.

3. Microkernel

Only the absolutely essential functions are kept in the kernel — minimal process and thread management, minimal memory management, and basic inter-process communication (IPC). Everything else (file systems, device drivers, networking) runs as user-level server processes that communicate with each other and with applications purely through message passing. Examples: Mach, QNX, MINIX.

- **Advantage:** Easier to extend and port to new hardware; more reliable and secure, because a failure in a non-essential service (e.g., a file-system server) does not crash the whole system, only that service; smaller trusted computing base.
- **Disadvantage:** Performance overhead due to the increased number of user-mode/kernel-mode switches and message-passing calls needed to perform what would be a single function call in a monolithic kernel.

4. Modules (Loadable Kernel Modules)

Uses an object-oriented-like approach: the kernel has a set of core components and communicates with other services through pre-defined, well-known interfaces; new services (drivers, file systems) can be added by dynamically **loading modules** into the kernel at run time, without recompiling or rebooting. Examples: Linux, Solaris.

- **Advantage:** Flexible — functionality can be loaded/unloaded on demand; performance close to a monolithic kernel since modules run in kernel space and call each other directly once loaded, without message-passing overhead.
- **Disadvantage:** Because modules execute in kernel space, a faulty or malicious module can still corrupt the kernel or crash the whole system — weaker isolation than a microkernel.

5. Hybrid Systems

Most modern production operating systems do not follow one pure design — they combine approaches for a pragmatic balance of performance and modularity.

- **Linux / Solaris:** Monolithic core enhanced with dynamically Loadable Kernel Modules.
- **Windows:** Largely monolithic with a layered, module-like organisation internally.
- **macOS / iOS:** Hybrid “XNU” kernel — a Mach microkernel (IPC, scheduling, basic memory management) with BSD UNIX components layered on top for the POSIX API, file systems, and networking.
- **Android:** Modified Linux kernel at the base, with an application framework and runtime layered above it.

Note / Assumption: Silberschatz classifies the layered, microkernel, module, and hybrid designs together under “Operating-System Structure” (Ch. 2). If your syllabus / teacher expects only Monolithic, Layered, Microkernel and Modular in the answer, mentioning Hybrid systems as a bonus fifth category (with real OS examples) is still valid and scores extra credit.

QUESTION 2

CO2 • 10 Marks

2. (a) Consider the following process snapshot: (table of Process No., AT (Arrival Time), and Burst Time as CPU Burst / I/O Burst / CPU Burst). If the CPU scheduling policy is Priority Scheduling, calculate the average waiting time and average turnaround time. (Lower number means higher priority).

Process	AT	CPU Burst 1	I/O Burst	CPU Burst 2
P1	0	1	5	3
P2	2	3	3	1
P3	3	2	3	1

ANSWER

Note / Assumption: The table gives no explicit numeric priority column. The question's own hint — "lower number means higher priority" — is taken to mean that the **Process number itself is the priority rank**: P1 is highest priority, P2 next, P3 lowest. Scheduling is taken as **non-preemptive priority** (the standard default when a question does not explicitly say "preemptive"), and each I/O burst is assumed to run on a dedicated I/O channel that works in parallel with the CPU, so a process's I/O never blocks another process's I/O.

Step 1 — Scheduling trace

Each process needs the CPU twice (a CPU burst, then I/O, then a second CPU burst). At every point where the CPU is free, the ready process with the numerically lowest priority number (i.e., highest priority) is dispatched next, and — being non-preemptive — runs to the end of that burst once started.

CPU timeline:

```

0-1   P1 (burst 1)
1-2   CPU idle (no process ready – P2 arrives at t = 2)
2-5   P2 (burst 1) [P3 arrives at t = 3, waits]
5-7   P3 (burst 1) [P1 returns from I/O at t = 6, waits]
7-10  P1 (burst 2) [highest priority, runs as soon as CPU is free]
10-11 P2 (burst 2) [P2 back from I/O at t = 8; beats P3, which arrives at t = 10]
11-12 P3 (burst 2)
    
```

I/O timeline (parallel channel):

```

1-6   P1 in I/O
5-8   P2 in I/O
7-10  P3 in I/O
    
```

Step 2 — Completion, Turnaround and Waiting Time

Process	Arrival	Completion	Turnaround (Compl. – Arrival)	CPU + I/O Total	Waiting Time
P1	0	10	10	1+5+3 = 9	1
P2	2	11	9	3+3+1 = 7	2
P3	3	12	9	2+3+1 = 6	3

Waiting time for a process = Turnaround time – (sum of its CPU bursts) – (sum of its I/O time). e.g. for P1: $10 - (1+3) - 5 = 1$.

Average Turnaround Time = $(10 + 9 + 9) / 3 = 28/3 \approx 9.33$ time units

Average Waiting Time = $(1 + 2 + 3) / 3 = 2$ time units

2. (b) Five batch jobs, A through E, arrive at a computer centre at essentially the same time. They have an estimated running time of 11, 6, 2, 4 and 8 minutes respectively. Their (externally defined) priorities are 3, 5, 2, 1 and 4 respectively, with 5 being the higher priority. For each of the following scheduling algorithms, determine the turnaround time for each process and the average turnaround time for all jobs. Ignore process switching overhead. Assume only one job runs on the CPU at a time and that all jobs are completely processor-bound (no I/O). (i) Round robin with a time quantum of 1 minute. (ii) Priority scheduling.

Job	A	B	C	D	E
Burst Time (min)	11	6	2	4	8
Priority (5 = highest)	3	5	2	1	4

ANSWER

(i) Round Robin, time quantum = 1 minute

All five jobs arrive together, so they are placed in the ready queue in the listed order A→B→C→D→E, and the CPU cycles through them 1 minute at a time, skipping any job that has already finished, until every job's remaining burst reaches 0.

Order in which jobs finish (simulated round by round):

C finishes first – after 4 full rounds of 5 jobs + partial → completes at $t = 8$ (needed only 2 quanta)
D finishes next – completes at $t = 17$ (needed 4 quanta)
B finishes next – completes at $t = 23$ (needed 6 quanta)
E finishes next – completes at $t = 28$ (needed 8 quanta)
A finishes last – completes at $t = 31$ (needed 11 quanta, and is the only job left for the final 3 minutes, so it runs uninterrupted at the end)

Job	Burst	Completion Time	Turnaround Time
A	11	31	31
B	6	23	23
C	2	8	8
D	4	17	17
E	8	28	28

Average Turnaround Time (Round Robin, $q = 1$) = $(31+23+8+17+28) / 5 = 107/5 = 21.4$ minutes

Why this pattern always holds: with RR and simultaneous arrivals, no job can ever finish more than $(n-1) = 4$ minutes after a shorter/equal job that is still competing with it, and the job with the largest burst is necessarily the last one left running alone — so it always finishes exactly at the sum of all burst times ($11+6+2+4+8 = 31$ here).

(ii) Priority Scheduling (non-preemptive; 5 = highest priority)

Since every job arrives at the same instant, preemptive and non-preemptive priority scheduling give an identical result here — the jobs simply run strictly in order of priority, highest first: **B (5) → E (4) → A (3) → C (2) → D (1)**.

CPU timeline:

0–6 B | 6–14 E | 14–25 A | 25–27 C | 27–31 D

Job	Priority	Burst	Completion Time	Turnaround Time
B	5	6	6	6
E	4	8	14	14
A	3	11	25	25
C	2	2	27	27
D	1	4	31	31

Average Turnaround Time (Priority Scheduling) = $(6+14+25+27+31) / 5 = 103/5 = 20.6$ minutes

Note / Assumption: Priority scheduling (20.6 min) beats Round Robin (21.4 min) here mainly because job D (burst 4, the highest priority) is serviced early instead of being repeatedly interrupted. This is a well-known style of exercise from Silberschatz Ch. 5 (CPU Scheduling) — the same set-up is also commonly extended to compare against FCFS and Shortest-Job-First, which is worth practising separately since SJF gives the lowest possible average turnaround time of any non-preemptive algorithm for a fixed batch of jobs.

QUESTION 3

CO1/CO2 • 10 Marks

3. (a) Explain the features of shell. Also differentiate between the following: (i) Internal Command vs. External Command (ii) System Call vs. Library Functions.

ANSWER

The Shell

A **shell** is a command-line interpreter — a user-level program that acts as the interface between the user and the operating-system kernel. It reads a command typed interactively or from a script, interprets it, and either executes it directly (if it is a built-in) or asks the kernel to create a new process to run it (if it is an external program). Examples: *bash*, *csch*, *ksh*, *zsh* on UNIX/Linux; *cmd.exe* and PowerShell on Windows.

- **Command execution:** parses and runs both built-in and external commands.
- **I/O redirection:** lets a command's input/output be redirected to/from files using `<`, `>`, `>>`.
- **Piping:** connects the output of one command to the input of another using `|`, letting several small utilities be chained into a pipeline.
- **Scripting:** supports variables, conditionals, loops and functions, allowing sequences of commands to be saved and re-run as shell scripts.
- **Environment & variables:** maintains and exports environment variables (`PATH`, `HOME`, etc.) that configure how programs run.
- **Wildcard / filename expansion (globbing):** expands patterns like `*.txt` into matching filenames before a command runs.
- **Job control:** lets processes be run in the foreground or background (`&`), and be suspended, resumed or killed using signals.
- **Interactive conveniences:** command history, aliases, tab-completion, a customisable prompt.

(i) Internal Command vs. External Command

Aspect	Internal (Built-in) Command	External Command
What it is	Part of the shell program itself.	A separate executable file stored on disk (e.g., in <code>/bin</code> , <code>/usr/bin</code>).
Execution	Runs inside the shell's own process — no new process is created.	The shell must <i>fork</i> a child process and <i>exec</i> the program file.
Speed	Faster — no process-creation overhead.	Slower — incurs the cost of creating a new process.
Availability	Always available the moment the shell starts.	Available only if the program file exists and is found on <code>PATH</code> .
Examples	<code>cd</code> , <code>pwd</code> , <code>echo</code> , <code>exit</code> , <code>set</code> , <code>export</code> , <code>history</code>	<code>ls</code> , <code>cp</code> , <code>grep</code> , <code>cat</code> , <code>gcc</code> , <code>python</code>

(ii) System Call vs. Library Function

Aspect	System Call	Library Function
Provided by	The operating-system kernel.	A programming-language / runtime library (e.g., the C standard library).
Mode of execution	Causes a trap / software interrupt that switches the CPU from user mode to kernel mode.	Executes entirely in user mode (unless it internally calls a system call).
Overhead	Higher — involves a mode switch and kernel-side validation.	Lower, unless it wraps a system call internally.

Aspect	System Call	Library Function
Portability	OS-specific — interface differs across operating systems.	Often portable across operating systems, since the library hides the OS-specific system call underneath.
Examples	open(), read(), write(), fork(), exec(), wait()	printf() (uses write() internally), malloc() (uses brk()/mmap()), strcpy() (pure user mode, no system call)

In short: every use of a system call ultimately asks the kernel to do something on the program's behalf (privileged operations such as file I/O, process creation, memory allocation from the OS); a library function is simply ordinary user-mode code that a programmer calls for convenience, and which may or may not use a system call underneath.

3. (b) Consider a system running I/O bound tasks and a CPU bound task. Assume the I/O bound tasks issue an I/O operation once for every ms of CPU computing, and each I/O operation takes 10 ms to complete. Also assume the context-switch overhead is 1 ms and all processes need 2 ms (per Round-Robin cycle) to complete their turn. Describe the CPU utilisation for a round-robin scheduler when the time quantum is 1 ms.

ANSWER

Note / Assumption: This is a version of a well-known exercise from Silberschatz Ch. 5 (CPU Scheduling), where “all the processes need 2 ms to complete” each RR cycle matches exactly 1 ms quantum + 1 ms context-switch overhead computed below — i.e., it is telling us the length of one full RR turn, not a separate new value.

Reasoning

Each I/O-bound task needs only **1 ms of CPU time** before it must perform an I/O operation (10 ms long). Since the Round-Robin time quantum is also exactly 1 ms, every process is guaranteed to use its *entire* quantum doing useful computation and then voluntarily leave the CPU to start its I/O — it never gets pre-empted mid-burst. But every time the scheduler switches from one process to the next, it must pay a fixed **context-switch cost of 1 ms**, during which the CPU does no useful work at all.

So one complete Round-Robin “turn” for any process looks like:

One RR cycle:

[1 ms context switch (overhead)] + [1 ms quantum (useful work)] = 2 ms total

Out of every 2 ms of wall-clock time, only 1 ms is spent doing real work for the process; the other 1 ms is pure overhead. Hence:

CPU Utilisation = (time doing useful work) / (total time per cycle) = 1 ms / (1 ms + 1 ms) = 1/2 = **50%**

General formula to remember: CPU Utilisation = Time Quantum / (Time Quantum + Context-Switch Overhead). This is the key idea examiners are testing: making the time quantum too small relative to the context-switch cost wastes a large fraction of the CPU on overhead instead of useful work — which is exactly why choosing an appropriate time quantum is an important Round-Robin design decision. (As a point of comparison, if the quantum were increased to, say, 10 ms with the same 1 ms overhead, utilisation would rise to 10/11 ≈ 90.9%, at the cost of worse response time for interactive/I/O-bound processes.)

QUESTION 4

CO1/CO2 • 10 Marks

4. (a) Define thread. Explain how a process differs from a thread.

ANSWER

Thread

A **thread** is the basic unit of CPU utilisation within a process — often called a *lightweight process*. A thread has its own thread ID, program counter, register set and stack, but it **shares** its code section, data section, heap, and OS resources such as open files and signals with every other thread belonging to the same process. A process with a single thread is single-threaded; if it has multiple threads of control it is multi-threaded (e.g., a web browser process typically has separate threads for rendering, network fetches and the user interface, all sharing the same address space).

Process

A **process** is a program in execution. It is an independent unit of resource ownership — it has its own private virtual address space (code, data, heap, stack), its own set of open files and signal handlers, and at least one thread of execution running within it, and is represented in the OS by a Process Control Block (PCB).

Process vs Thread — Key Differences

Aspect	Process	Thread
Definition	A program in execution; an independent, self-contained unit.	A single flow of control (unit of CPU scheduling) inside a process.
Address space / memory	Each process has its own separate address space.	All threads of a process share the same address space (code, data, heap, open files).
Private per-unit state	Entire memory image, file descriptors, etc.	Only its own thread ID, program counter, register set and stack.
Creation overhead	Heavy — OS must allocate a fresh address space and PCB.	Lightweight — only a small thread control block is allocated.
Context-switch cost	High — also involves switching the memory-management context (e.g., page tables, TLB flush).	Low — no address-space switch needed among threads of the same process.
Communication	Requires inter-process communication (pipes, message queues, shared memory) — relatively expensive.	Direct, since memory is already shared — relatively cheap, but needs synchronisation (locks, semaphores) to avoid races.
Fault isolation	A crash in one process does not directly affect another process.	A crash/error in one thread (e.g., invalid memory access) can bring down the entire process, including all its other threads.
Resource ownership	Owns resources (memory, files, etc.).	Uses resources owned by its parent process.

4. (b) Write short notes on the following: (i) Dispatcher (ii) Context Switch.

ANSWER

(i) Dispatcher

The **dispatcher** is the OS module that actually hands control of the CPU to the process chosen by the short-term (CPU) scheduler. It is invoked on every context switch and performs three tasks:

- Switching context — saving the state of the currently running process and loading the state of the newly selected one.
- Switching to user mode — since the selected process must run in unprivileged mode.
- Jumping to the proper location in the user program to resume its execution at the correct instruction (restoring the saved program counter).

The dispatcher must be as fast as possible, since it runs on every process switch; the time it takes is called **dispatch latency**, and it is pure scheduling overhead (no useful work is done for any process while the dispatcher itself is running).

(ii) Context Switch

A **context switch** is the act of saving the state (“context”) of the CPU for the process currently executing — typically stored in its Process Control Block (PCB): program counter, CPU registers, memory-management information (page-table pointers), process state, etc. — and loading the previously saved context of the next process to run.

- **When it happens:** on an interrupt, a system call that blocks, an I/O request, or whenever the scheduler decides to switch the CPU to a different process/thread.
- **Cost:** pure overhead — the system performs no useful computation while switching. Its exact cost is highly hardware-dependent (e.g., how many registers must be copied, whether the memory-management unit needs its page tables/TLB reloaded and flushed).
- **Hardware support:** some processors provide multiple sets of registers, so switching just means changing a pointer to the currently active register set — making context switches considerably faster.
- **Thread vs process context switch:** switching between threads of the *same* process is cheaper than switching between different processes, since the address space (and hence the memory-management context) does not need to change.

QUESTION 5

CO1/CO2 • 10 Marks

5. (a) Differentiate between Multilevel Queue scheduling vs. Multilevel Feedback Queue scheduling.

ANSWER

Multilevel Queue (MLQ) Scheduling

The ready queue is partitioned into several **separate queues**, e.g. a *foreground* (interactive) queue and a *background* (batch) queue. Each process is **permanently assigned** to one queue, generally based on some property of the process such as memory size, process type, or priority. Each queue can have its own scheduling algorithm (say Round Robin for the foreground queue and FCFS for the background queue), and scheduling must additionally happen *between* the queues themselves — either fixed-priority (always serve the foreground queue first, which risks starving the background queue), or time-slicing (e.g., 80% of CPU time to foreground, 20% to background).

Multilevel Feedback Queue (MLFQ) Scheduling

MLFQ generalises MLQ by allowing processes to **move between queues** based on their observed behaviour, instead of being permanently fixed to one queue. A process that uses too much CPU time is **demoted** to a lower-priority queue; a process that has been waiting too long in a lower queue can be **promoted** (aging) to prevent starvation. This way CPU-bound and I/O-bound processes are automatically separated and treated appropriately without the OS having to classify them in advance. An MLFQ scheduler is defined by: the number of queues, the scheduling algorithm for each queue, the method used to decide when to upgrade a process, the method used to decide when to demote a process, and the method used to decide which queue a process enters when it first needs service.

Aspect	Multilevel Queue	Multilevel Feedback Queue
Queue assignment	Fixed / permanent for the life of the process.	Dynamic — a process can move between queues.
Adaptivity	Cannot adapt to a process's changing CPU-burst behaviour.	Adapts automatically by observing burst behaviour.
Starvation	Low-priority queues can starve under fixed-priority inter-queue scheduling.	Starvation is mitigated using aging (gradual promotion).
Complexity	Simpler to design and implement.	The most general and flexible CPU scheduling algorithm, but also the most complex — needs several parameters tuned for the system.
Typical use	Systems where a process's category is known in advance.	General-purpose OS schedulers (historically used in UNIX/Windows variants).

5. (b) Describe the functions of Operating System in concern with: (i) Process Management (ii) Memory Management (iii) File Management (iv) I/O Management.

ANSWER

(i) Process Management

- Creating and deleting both user and system processes.
- Suspending and resuming processes as needed.
- Providing mechanisms for process synchronisation (so cooperating processes coordinate correctly).
- Providing mechanisms for inter-process communication (shared memory, message passing).
- Providing mechanisms for deadlock handling (prevention, avoidance, detection and recovery).

(ii) Memory Management

- Keeping track of which parts of memory are currently in use, and by which process.
- Deciding which processes (or parts of them) to load into memory when space becomes available, and which to move out.
- Allocating and de-allocating memory space dynamically as processes request and release it.
- Enforcing memory protection, so one process cannot read or corrupt another's memory.
- Supporting virtual-memory techniques such as paging and segmentation, so processes can use more (logical) memory than is physically installed.

(iii) File Management

- Creating and deleting files and directories.
- Providing primitives to manipulate files and directories (read, write, rename, move, etc.).
- Mapping files onto secondary-storage locations.
- Backing up files on stable (non-volatile) storage media.
- Providing access control so that only authorised users/processes can access a given file, with defined permissions (read / write / execute).

(iv) I/O Management

- Providing a general device-driver interface, hiding the peculiarities of specific hardware devices from user programs (*device independence*).
- Managing device drivers for the wide variety of specific I/O hardware (disks, printers, network cards, etc.).
- Using buffering, caching and spooling to smooth out and speed up I/O, and to let a device be shared even though it may fundamentally support only one job at a time (e.g., a printer via spooling).
- Handling interrupts raised by I/O devices on completion of an operation, and reporting errors reliably back to the requesting process.

QUICK REVISION CHEAT-SHEET

Scheduling formulas

- **Turnaround Time** = Completion Time – Arrival Time
- **Waiting Time** = Turnaround Time – Burst Time (subtract I/O time too, if any)
- **Response Time** = Time of first CPU allocation – Arrival Time
- **CPU Utilisation (Round Robin)** = Time Quantum / (Time Quantum + Context-Switch Overhead)
- Among non-preemptive algorithms with a fixed batch of jobs, **SJF (Shortest Job First)** minimises average waiting time.

Scheduling algorithms at a glance

Algorithm	Preemptive ?	Key idea	Weakness
FCFS	No	Runs jobs strictly in arrival order.	Convoy effect — short jobs stuck behind long ones.
SJF / SRTF	No / Yes	Shortest burst (remaining time) runs first.	Needs burst-time prediction; can starve long jobs.
Priority	No / Yes	Highest-priority ready job runs.	Can starve low-priority jobs (fixed by aging).
Round Robin	Yes	Each job gets a fixed time quantum, then goes to queue tail.	Quantum too small → overhead; too large → behaves like FCFS.
Multilevel Queue	Varies	Separate queues by process category; each queue has its own policy.	Fixed queue assignment; can starve lower queues.
Multilevel Feedback Queue	Yes	Like MLQ, but processes move between queues based on behaviour.	Complex to configure and tune.

Core definitions to have exam-ready

- **Process:** a program in execution, with its own address space and at least one thread.
- **Thread:** a lightweight unit of CPU scheduling within a process, sharing the process's code/data/resources.
- **Context Switch:** saving the state of one process/thread and loading another's, so the CPU can switch between them.
- **Dispatcher:** the module that gives the CPU to the process chosen by the scheduler.
- **System Call:** a request from a user program to the kernel for a privileged service, via a user-mode → kernel-mode trap.
- **Kernel:** the core of the OS, running in privileged mode, structured as Monolithic, Layered, Microkernel, Modular, or Hybrid.
- **Multilevel Feedback Queue:** the most general CPU-scheduling algorithm — queues with movable, adaptive process assignment.

Reference for further reading: Abraham Silberschatz, Peter Baer Galvin, Greg Gagne, **Operating System Concepts**, 9th Edition — Ch. 1 (Introduction), Ch. 2 (Operating-System Structures), Ch. 3 (Processes), Ch. 4 (Threads & Concurrency), and Ch. 5 (CPU Scheduling).